DP-LMI package
User Manual

Version v0.1.0

July 2026

Abstract

This manual describes the current public behavior of the DP-LMI package. The present implementation provides the `dpbase` class and shared validation tests for cell-local Bernstein coefficient storage on tensor grids. Solver-facing DP-LMI objects such as `dpmat`, `dpvar`, and `dplmi` are planned package layers, but they are not yet public APIs in this repository version.

Contents

1 Installation And Verification

1.1 Add The Package To The MATLAB Path

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

```
projectRoot = "path/to/Differential Parameter—Dependent Linear Matrix Inequality";
addpath(genpath(projectRoot));
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local development from the repository root, the shorter form is enough:

```
addpath(pwd)
```

1.2 Run The Current Test Suite

The current verification entry point runs the non-SDP tests for `+helper` and `@dpbase`.

```
results = tests.run_all;
table(results)
```

Expected result: all currently implemented tests pass. At the time this manual was written, the suite contained 30 tests covering helper validation and `dpbase` construction, grid metadata, local storage, label ordering, and rate-bound validation.

The command above does not verify YALMIP modeling, SDP solver calls, relaxation lemma assembly, or DP-LMI feasibility workflows because those public layers are not implemented in this version.

2 Quick Start

`dpbase` stores one flat Bernstein coefficient cell for each physical cell of a tensor grid. The default constructor creates zero numeric coefficient payloads; it does not sample a function at grid nodes.

2.1 A Scalar Grid

```
obj = dpbase({[0 1 3]}, [2 3], 1);

obj.MatrixSize
obj.Degree
obj.GridInfo.Bounds
obj.npar()
obj.ncell()
obj.ncoeff()
size(obj)
obj.cells()
obj.lbls()
```

Expected structural outputs:

```
MatrixSize =
     2     3

Degree =
     1

Bounds =
     0     3

npar()   -> 1
ncell()  -> 2
ncoeff() -> 2
size(obj)-> [2 3]

cells() =
     1
     2

lbls() =
     0
     1
```

There are two physical intervals, $[0, 1]$ and $[1, 3]$. Because the degree is one and the grid is one-dimensional, each physical cell stores two local Bernstein coefficients.

2.2 Access Local Coefficients

```
c1 = obj.coeffs(1);
c2 = obj.coeffs(2);
```

Expected result: both `c1` and `c2` are `1x2` cell arrays. Each entry is a numeric zero matrix of size `2x3`. The first cell corresponds to local labels 0 and 1 on the interval $[0, 1]$; the second cell uses the same local labels on the interval $[1, 3]$.

2.3 A Tensor Grid

```
obj = dpbase({[0 1 2], [10 20 30]}, [1 1], 1);

obj.GridInfo.Points
obj.cells()
obj.lbls()
obj.coeffs([2 1])
```

Expected structural outputs:

```
GridInfo.Points =
     0    10
     0    20
     0    30
     1    10
     1    20
     1    30
```

```
     2    10
     2    20
     2    30

cells() =
     1     1
     1     2
     2     1
     2     2

lbls() =
     0     0
     0     1
     1     0
     1     1
```

The first parameter index varies more slowly than the second. This row order is used consistently for physical cells and local Bernstein labels.

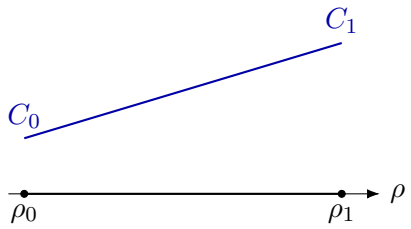
3 Bernstein Storage Background

3.1 One Cell, Local Coordinates

On one physical interval $[\rho_0, \rho_1]$, a local coordinate $\alpha \in [0, 1]$ can represent a degree-m Bernstein polynomial as

$$P(\alpha) = \sum_{i=0}^{m} C_i B_i^m(\alpha), \qquad B_i^m(\alpha) = \binom{m}{i} \alpha^i (1-\alpha)^{m-i}.$$

For matrix-valued data, each coefficient C_i is a matrix with the same payload size. `dpbase` stores those matrices as a flat cell array.



one physical cell, two degree-1 local labels

3.2 Tensor-Product Labels

For ℓ parameter dimensions and degree m, each local label is an ℓ-tuple in $\{0, \ldots, m\}^\ell$. The number of local coefficients per physical cell is

$$(m+1)^\ell.$$

For example, $\ell = 2$ and $m = 1$ gives labels

$$(0,0),\ (0,1),\ (1,0),\ (1,1),$$

which is exactly the order returned by `obj.lbls()`.

3.3 Products And Degree Growth

When two Bernstein polynomials are multiplied on the same physical cell, their local labels add and the result has the sum of the operand degrees. The package contains private implementation utilities for this binomial-scaled convolution and degree elevation. These utilities are internal; users should inspect coefficient ordering through public methods such as `lbls`, `cells`, `ncoeff`, and `coeffs`.

4 `dpbase` Reference

4.1 Purpose

`dpbase` is the shared coefficient-backed storage class for gridded DP-LMI objects. In this repository version it is also the only implemented public class. It stores tensor-grid metadata, matrix payload size, Bernstein degree, nested physical-cell coefficient data, and rate-dependence metadata.

4.2 Syntax

```
obj = dpbase(gridVectors, matrixSize, degree)
obj = dpbase(gridVectors, matrixSize, degree, localValues)
obj = dpbase(gridVectors, matrixSize, degree, localValues, Name=Value)
```

4.3 Inputs

Input	Description
`gridVectors`	Nonempty cell array of grid node vectors. Each vector must be finite, real, strictly increasing, numeric, and contain at least two nodes.
`matrixSize`	`1x2` positive integer vector giving the row and column size of every coefficient payload.
`degree`	Nonnegative integer scalar Bernstein degree.
`localValues`	Optional nested cell payload. If omitted or empty, `dpbase` creates zero numeric coefficient matrices for every local coefficient of every physical cell.

4.4 Name-Value Options

Option	Description
`IsContinuous`	Logical metadata flag. `dpbase` records the flag but does not enforce boundary sharing itself. Future subclasses own continuity construction.
`ContainsDecision`	Logical metadata flag for future decision-variable subclasses.
`HasRateDependence`	Logical metadata flag. If true, nonempty `RateBounds` must be supplied.
`RateBounds`	Empty, or an $\ell \times 2$ finite real matrix with lower bounds in column one and upper bounds in column two. The number of rows must match the number of parameter dimensions.
`SourceSummary`	String metadata describing the coefficient source. The default is `"coefficient-backed"`.

4.5 Properties

All public properties have private set access. Users can inspect them but cannot assign to them after construction.

Property	Meaning
`GridInfo`	Struct with fields `Vectors`, `Points`, `Bounds`, and `NumNodes`.
`MatrixSize`	Matrix payload size stored as a `1x2` vector.
`Degree`	Bernstein degree.
`LocalValues`	Nested physical-cell storage. A tensor grid uses nested brace access, for example `LocalValues{i1}` followed by `{i2}` and later dimensions; each leaf stores a flat coefficient cell.
`IsContinuous`	Continuity metadata flag.
`ContainsDecision`	Decision-dependence metadata flag.
`HasRateDependence`	True when rate metadata or rate-affine payloads are present.
`RateBounds`	Empty or an $\ell \times 2$ finite real rate-bound table.
`SourceSummary`	String metadata for the coefficient source.

4.6 Methods

Method	Behavior
`size(obj)`	Return the matrix payload size. `size(obj,dim)` follows ordinary matrix behavior for dimensions beyond two by returning one.
`npar(obj)`	Return the number of parameter dimensions.
`ncell(obj)`	Return the number of physical tensor-grid cells.
`ncoeff(obj)`	Return the number of local Bernstein coefficients per cell.
`cells(obj)`	Enumerate physical-cell subscripts in flat row order.
`lbls(obj)`	Enumerate local Bernstein labels in flat row order.
`coeffs(obj,cellSubs)`	Return the flat coefficient cell for one physical cell. `cellSubs` can be a numeric row vector or a cell array with one physical-cell index per parameter.

4.7 Custom Local Values

For a one-dimensional grid with two physical cells and degree two, each physical cell must store three local coefficients.

```
localValues = {{11, 12, 13}, {21, 22, 23}};
obj = dpbase({[0 1 2]}, [1 1], 2, localValues);

obj.coeffs(2)
obj.coeffs({1})
```

Expected output:

```
obj.coeffs(2)  -> {21, 22, 23}
obj.coeffs({1})-> {11, 12, 13}
```

For a tensor grid, physical cells are nested one parameter dimension at a time. The storage contract is `LocalValues{i1}{i2}...`, not `LocalValues{i1,i2,...}`.

```
mkCoeff = @(offset) {offset+1, offset+2, offset+3, offset+4};
localValues = {
    {mkCoeff(110), mkCoeff(120)}, ...
    {mkCoeff(210), mkCoeff(220)}
    };

obj = dpbase({[0 1 2], [10 20 30]}, [1 1], 1, localValues);
obj.coeffs([2 1])
```

Expected output:

```
ans =
  1x4 cell array
    {[211]}    {[212]}    {[213]}    {[214]}
```

4.8 Rate Metadata

`dpbase` validates rate bounds and rate-affine coefficient payloads, but it does not enumerate rate vertices or assemble LMIs. That solver-facing work is reserved for a future `dplmi` layer.

```
obj = dpbase({[0 1], [10 20]}, [1 1], 0, [], ...
    RateBounds=[-1 1; -2 2]);

obj.HasRateDependence
obj.RateBounds
obj.coeffs([1 1])
```

Expected result: `HasRateDependence` is true, `RateBounds` is the given `2x2` table, and the single local coefficient remains the numeric zero payload.

Rate-affine payloads use a scalar struct with fields `Constant` and `Rate`. The `Rate` field must be a cell array with one matrix per parameter dimension.

```
payload = struct("Constant", eye(2), "Rate", {{ones(2), 2*ones(2)}});
localValues = {{{payload}}};
obj = dpbase({[0 1], [10 20]}, [2 2], 0, localValues, ...
    RateBounds=[-1 1; -2 2]);
```

The stored coefficient represents the compact form

$$C(\dot{\rho}) = C_0 + C_1\dot{\rho}_1 + C_2\dot{\rho}_2,$$

but `dpbase` only stores and validates this payload. It does not solve or test any inequality involving $\dot{\rho}$.

5 Validation And Error Boundaries

The constructor rejects malformed inputs early and uses domain-specific error identifiers. The following examples are expected to error:

```
dpbase([0 1], [1 1], 0)                % dpbase:InvalidGrid
dpbase({[0 1 1]}, [1 1], 0)            % dpbase:InvalidGridVector
dpbase({[0 1]}, [1 0], 0)              % dpbase:InvalidMatrixSize
dpbase({[0 1]}, [1 1], -1)              % dpbase:InvalidDegree
dpbase({[0 1]}, [1 1], 0, [], ...
    HasRateDependence=true)            % dpbase:InvalidRateBounds
```

Plain coefficient payloads at the `dpbase` layer must be finite, real, numeric matrices matching `matrixSize`. Symbolic YALMIP payloads are not accepted by `dpbase`; they belong in future subclasses that can define symbolic algebra and solver assembly coherently.

6 Current Limits

This repository version intentionally has a narrow public surface.

- `dpbase` and `helper.chk` are implemented and tested.
- `dpmat`, `dpvar`, and `dplmi` are planned package layers, but they are not present as public class folders in this version.
- There is no public `dpdiff` class. Future derivative behavior is expected to belong to `dpvar`.
- YALMIP modeling, SDP solver invocation, finite rate-vertex assembly, and relaxation lemma workflows are not public behavior yet.
- Private Bernstein utilities support internal tests and implementation, but they are not user-facing functions.

7 Style References Used For This Manual

This manual follows the organization style common in mature MATLAB and control-toolbox documentation: installation first, a small runnable example, conceptual background, then API reference with syntax, arguments, examples, validation rules, and current limits. Local LPVTools, ROLMIP, and ROMULOC documentation were used as style references only. Their APIs and text are not dependencies of this package and are not copied here.